Egyptian Military Academy-NTI

Digilians – Semester 1, 2026

Student Course Management System

C++ Console Application – Individual Project Report

Field	Details
Student Name	Amira Hamada Mohamed Deef
Student ID	12217
Course	SW-101: Foundation of Software Programming
Instructor	Dr. Mostafa Ashraf
Institution	Egyptian Military Academy – Digilians
Semester	Semester 1, 2026
GitHub Repository	https://github.com/AmiraDeef/Student-Courses-Management

Table of Contents

Table of Contents 1
1. Introduction 1
2. Detailed Project Description 1
2.1 Main Features 1
2.2 Data Persistence 1
2.3 Predefined Course List 1
3. Project Beneficiaries 1
3.1 Students 1
3.2 Academic Administrators 1
3.3 Instructors 1
3.4 Developer (Learner) 1
4. Detailed Analysis 1
4.1 Problem Analysis 1
4.2 Design Decisions 1
4.3 Class Responsibilities 1
5. Task Breakdown Structure 1
6. Time Plan 1
7. System Architecture and Design 1
7.1 Architecture Overview 1
7.2 OOP Concepts Applied 1
7.3 STL Usage 1
7.4 File Format 1
7.5 ID Management 1
8. Testing Scenarios and Results 1
9. End-User Guide 1
9.1 Requirements 1
9.2 Compiling with Visual Studio 1
9.3 Compiling with G++ (Command Line) 1
9.4 Using the Application 1
9.5 Notes for the User 1
10. Sample Outputs 1
10.1 Adding a Student 1
10.2 Enrolling in a Course 1

10.3 Enrolling in a Duplicate Course 1
10.4 Displaying All Students 1
10.5 Sorting by GPA 1
11. Conclusion 1
12. References 1

1. Introduction

This report presents the design and implementation of a Student Course Management System developed as part of the SW-101 Foundation of Software Programming course.
The project is a C++ console application that applies basic programming concepts such as Object-Oriented Programming (OOP) and the use of Standard Template Library (STL). The program is designed using a simple and clear structure to make it easy to understand and use.
The system allows the user to manage student data through a menu-driven interface. The user can add new students, remove existing ones, search for a student by ID, and display all students. In addition, each student can be enrolled in courses, and their courses can be viewed easily.
The project was developed individually using C++ and Visual Studio. The main focus of this project is to practice OOP concepts like classes, encapsulation, and inheritance, as well as using STL containers such as vector and set.
This report includes the problem description, system design, and a brief explanation of how the program works.

2. Detailed Project Description

The Student Course Management System is a console-based application written in C++. It manages a list of students, where each student has a unique auto-generated ID, a name, a GPA, and a set of enrolled courses. The system provides eight operations accessible through a numbered menu.

2.1 Main Features

- Add a new student by entering their name and GPA. The system assigns a unique ID automatically starting from 1001.
- Remove an existing student by entering their ID.
- Search for a student by ID and display their name and GPA.
- Display all students currently stored in the system, along with their enrolled courses.
- Enroll a student in one of six predefined courses by entering the student ID and the course ID.
- Display all courses a specific student is enrolled in.
- Sort all students by GPA in descending order and display the sorted list.
- Exit the program, which automatically saves all data to a text file before closing.

2.2 Data Persistence

One of the key features implemented beyond the basic requirements is file handling. When the program starts, it loads all previously saved student data from a file called students.txt. When the user exits or adds a student, the data is saved back to the file. This prevents data loss between sessions and makes the system behave more like a real-world application.

2.3 Predefined Course List

Instead of allowing users to type course names manually, the system uses a predefined list of six courses stored as Course objects. Each course has a unique ID and a name. The available courses are:

- 101 – Math
- 102 – OOP
- 103 – Data Structure
- 104 – Operating Systems
- 105 – Frontend
- 106 – Backend

This design decision reduces input errors and makes the enrollment process straightforward

3. Project Beneficiaries

This project, while built as an educational exercise, reflects a type of system that serves multiple stakeholders in an academic environment:

3.1 Students

Students benefit from having a centralized record of their enrolled courses and GPA. The search and display features allow a student to quickly check their information without browsing through large lists.

3.2 Academic Administrators

Administrative staff benefit from features like adding and removing students, enrolling students in courses, and sorting students by GPA. These operations reflect common administrative tasks in any educational institution.

3.3 Instructors

Instructors can use a system like this to view which students are enrolled in their courses and compare student GPAs, which helps in tracking academic performance.

3.4 Developer (Learner)

From a learning perspective, the primary beneficiary of this project is the developer. Building this system provided practical experience with OOP principles, STL containers, file I/O, and the process of designing a modular C++ program from scratch.

4. Detailed Analysis

4.1 Problem Analysis

The core problem is managing student and course data in a structured way. Before implementation, the following requirements were identified:

- Each student must have a unique identifier that persists even after the program is restarted.
- GPA must be validated to fall within the range of 0.0 to 4.0.
- A student should not be enrolled in the same course more than once.
- The system must handle invalid inputs without crashing.
- All data must be stored and retrieved from a file to support persistence.

4.2 Design Decisions

Several design decisions were made during the analysis phase:

- A static counter was used inside the Student class to auto-generate unique IDs. The counter starts at 1000 and increments by one for each new student. When loading from a file, the counter is restored to the highest existing ID to avoid conflicts.
- Courses are stored as a std::set inside each Student object. A set was chosen because it automatically prevents duplicate entries and keeps courses sorted by name alphabetically.
- The Management class holds all students in a std::vector, which allows indexed access, iteration, and the use of STL algorithms like std::sort and std::find_if.
- File data is stored in a comma-separated text format where each line represents one student with all their courses included in the same line.

4.3 Class Responsibilities

The application is divided into five classes, each with a clear responsibility:

Class	Responsibility
Person	Base class holding the name attribute and its getter method.
Student	Inherits from Person. Manages student ID, GPA, and enrolled courses. Handles adding and displaying courses.
Course	Holds course ID and name. Implements the less-than operator to allow storage in a set.
Management	Manages the collection of students. Provides all CRUD operations, sorting, enrollment, and file I/O.

Main	Entry point. Displays the menu, reads user input, and delegates all actions to the Management object.

5. Task Breakdown Structure

The project was broken down into the following major tasks:

#	Task	Description
1	Requirements Analysis	Read the assignment, identify required features, constraints, and bonus features to implement.
2	Class Design	Design the Person, Student, Course, and Management classes including attributes, methods, and inheritance structure.
3	UML Diagram	Draw the UML class diagram showing all classes, their attributes, methods, and relationships.
4	Core Implementation	Implement all classes and the Management methods: add, remove, search, display, enroll, sort.
5	File Handling	Implement saveToFile() and loadFromFile() to persist data between sessions using fstream and stringstream.
6	Input Validation	Add input validation for GPA range (0.0–4.0) and invalid menu choices to prevent crashes.
7	Menu System	Implement the interactive menu loop in main() with a switch-case handler.
8	Testing	Test all features manually: add, remove, search, enroll, sort, save, and load.
9	GitHub & README	Push code to GitHub and write a clear README file with setup instructions, design notes, and limitations.
10	Report Writing	Write the project report covering all required sections.

6. Time Plan

The project was completed over one week. The following Gantt chart shows the distribution of tasks across the seven days:

Task	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7
Requirements Analysis	✓						
Class Design		✓					
UML Diagram		✓					
Core Implementation			✓	✓			
File Handling				✓	✓		
Input Validation					✓		
Menu System			✓	✓			
Testing						✓	
GitHub & README						✓	
Report Writing							✓

Figure 1 – Gantt Chart showing task distribution over the project week

7. System Architecture and Design

7.1 Architecture Overview

The system follows a layered architecture. The main function acts as the presentation layer, handling user interaction through the menu. The Management class acts as the business logic layer, controlling all operations on student data. The Student, Person, and Course classes form the data layer, representing the actual entities being managed.

7.2 OOP Concepts Applied

The following OOP principles are demonstrated in the code:

- Inheritance: The Student class inherits from the Person class. The name attribute and getName() method are defined in Person and reused in Student without duplication.
- Encapsulation: All class attributes are private or protected. Access is provided only through public getter and setter methods. For example, Student.id, Student.gpa, and Student.courses are all private.
- Constructors: Each class defines a constructor to initialize its attributes. The Student constructor auto-increments a static counter to assign a unique ID.
- Operator Overloading: The Course class overloads the less-than operator (<) to allow Course objects to be stored in a std::set, which requires a comparison operator to maintain order.

7.3 STL Usage

Two STL containers are used:

- std::vector<Student> in the Management class to store all students. This allows dynamic resizing, iteration, and use of STL algorithms.
- std::set<Course> in the Student class to store enrolled courses. The set prevents duplicate enrollments and maintains courses in alphabetical order automatically.

The following STL algorithms from the <algorithm> header are used:

- std::find_if: Used to search for a student by ID in the vector, and to search for a course in the predefined course list.
- std::sort: Used to sort the student vector by GPA in descending order using a lambda comparator.

7.4 File Format

Each line in students.txt represents one student. The format is:

```
ID,Name,GPA,CourseID,CourseName,CourseID,CourseName,...
```

For example, a student named Ali with GPA 3.5 enrolled in Math and OOP would be stored as:

```
1001,Ali,3.5,101,Math,102,OOP
```

7.5 ID Management

Student IDs are managed through a static counter variable in the Student class. The counter starts at 1000 and is incremented before each new student is created, so the first student gets ID 1001. When loading from file, the system reads all existing IDs, finds the highest one, and restores the counter to that value. This ensures that newly added students after loading always receive IDs that are higher than any existing ones, preventing ID conflicts.

8. Testing Scenarios and Results

The following test cases were executed manually to verify the correctness of each feature. All tests were conducted by running the compiled application in a Windows console environment.

TC#	Test Case	Input	Expected Output	Result
TC1	Add a new student	Name: Amira, GPA: 3.8	Student added with auto-generated ID (e.g., 1001)	PASS
TC2	Add student with invalid GPA	Name: Sara, GPA: 5.0	Prompt to re-enter GPA; GPA must be between 0.0 and 4.0	PASS
TC3	Search for existing student	ID: 1001	Displays student name and GPA correctly	PASS
TC4	Search for non-existing student	ID: 9999	Displays: 'Student not found'	PASS
TC5	Enroll student in a course	Student ID: 1001, Course ID: 101	Math added to student's courses	PASS
TC6	Enroll in same course again	Student ID: 1001, Course ID: 101	Notice: Already enrolled in Math!	PASS
TC7	Remove existing student	ID: 1001	Student removed successfully	PASS
TC8	Remove non-existing student	ID: 9999	Displays: 'Student not found'	PASS
TC9	Sort students by GPA	Multiple students with different GPAs	Students listed in descending GPA order	PASS
TC10	Save and reload data	Add students, exit, restart program	All previously added students and courses are loaded correctly	PASS
TC11	Invalid menu choice	Enter: 99	Prompts to re-enter a valid number (0–7)	PASS
TC12	Non-numeric input at menu	Enter: abc	Clears input, prompts for valid number	PASS

Table 1 – Test cases and results

9. End-User Guide

9.1 Requirements

To compile and run this application, the following are required:

- A C++ compiler: GCC (via MinGW on Windows) or Microsoft Visual Studio
- Windows OS is recommended for Visual Studio users
- All source files must be present: Student_course_Managment.cpp, Student.cpp, Management.cpp, Course.cpp, Person.cpp, and their corresponding header files

9.2 Compiling with Visual Studio

1. Open Visual Studio.
2. Go to File > Open > Project/Solution.
3. Select the Student_course_Managment.sln file.
4. Press Ctrl + Shift + B to build the project.
5. Press Ctrl + F5 to run without debugging.

9.3 Compiling with G++ (Command Line)

Navigate to the folder containing all source files and run:

```
g++ -o StudentSystem Student_course_Managment.cpp Student.cpp Managment.cpp
Course.cpp Person.cpp
```

Then run the program:

```
./StudentSystem
```

9.4 Using the Application

When the program starts, it loads any existing student data from students.txt and then shows the main menu:

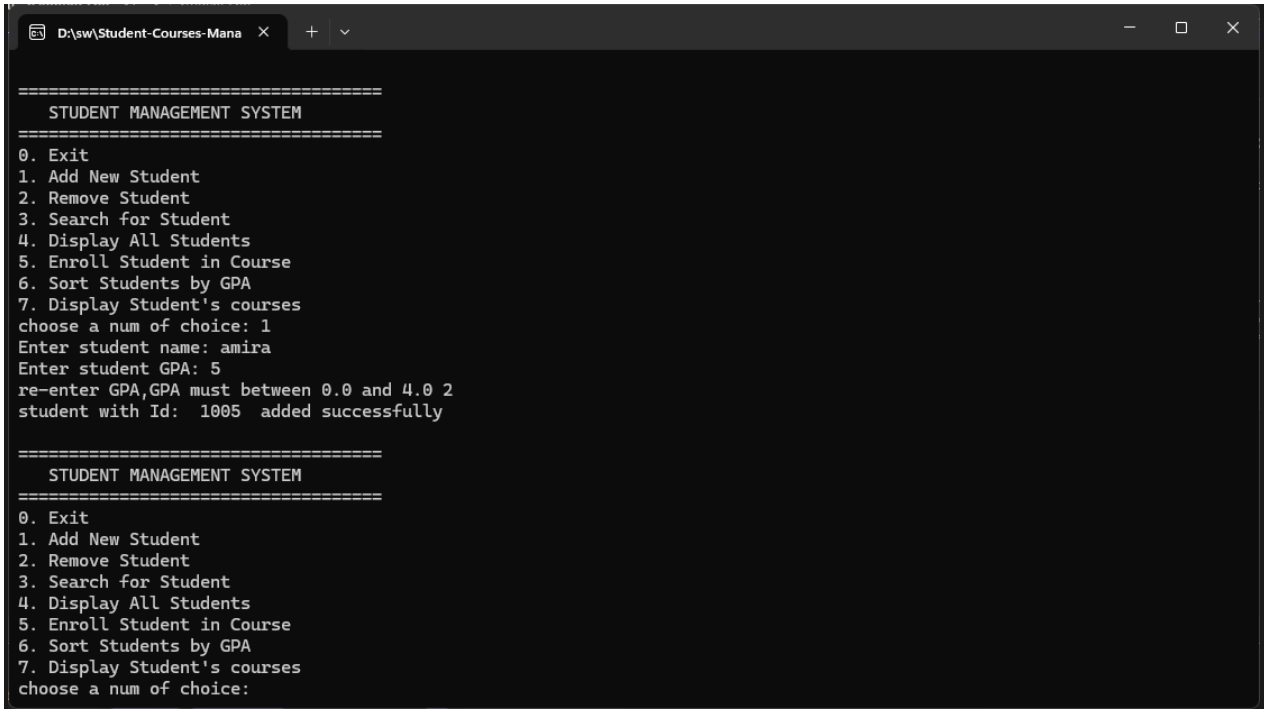
Enter the number corresponding to the action you want to perform. Follow the on-screen prompts for any required input.

9.5 Notes for the User

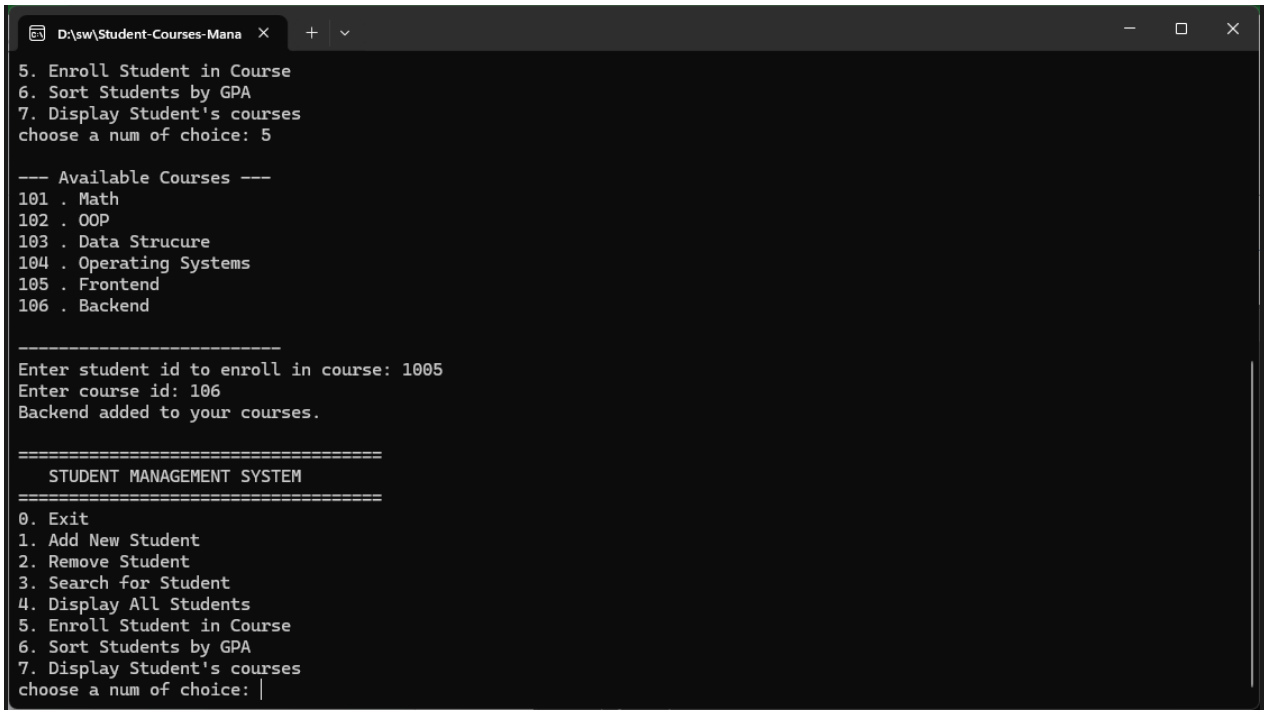
- Student IDs are assigned automatically. You do not need to enter an ID when adding a student.
- GPA must be a number between 0.0 and 4.0. Invalid values will prompt you to re-enter.
- When enrolling a student in a course, the system will first display all available courses with their IDs for reference.
- All changes are saved automatically when you add a student or choose Exit (option 0).
- If you close the program using the window's X button instead of option 0, data may not be saved.

10. Sample Outputs

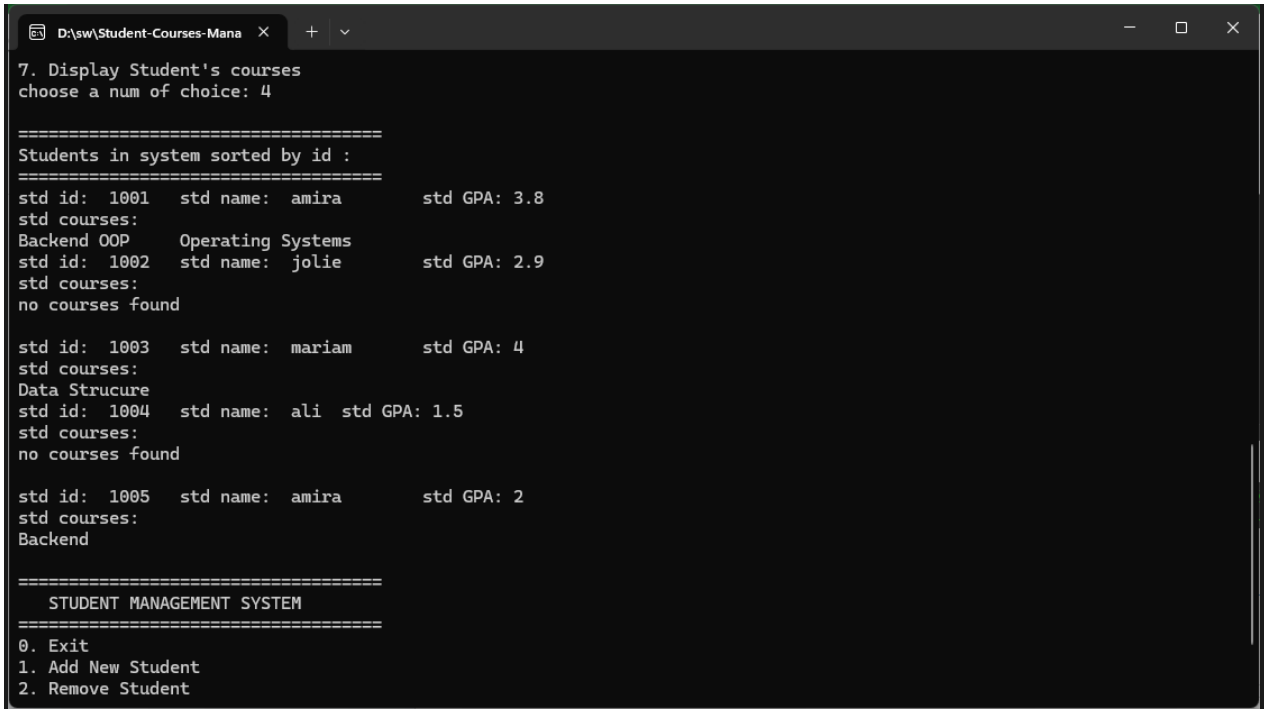
10.1 Adding a Student



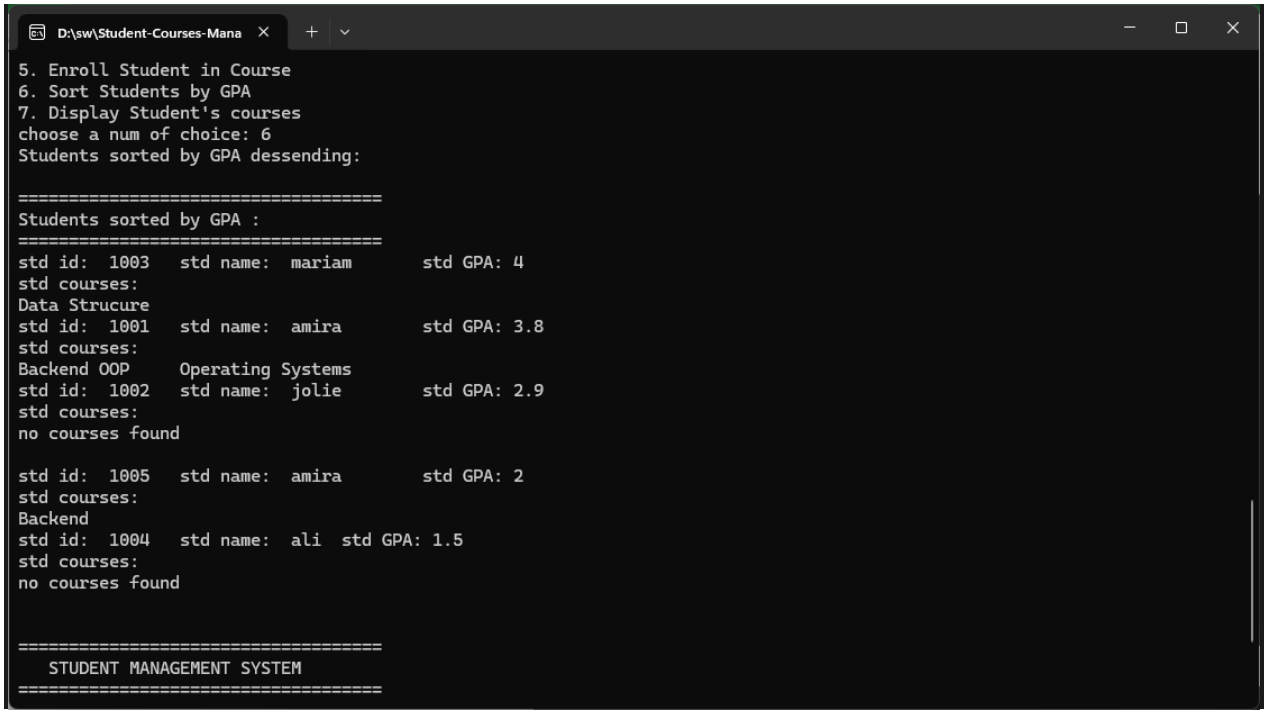
10.2 Enrolling in a Course



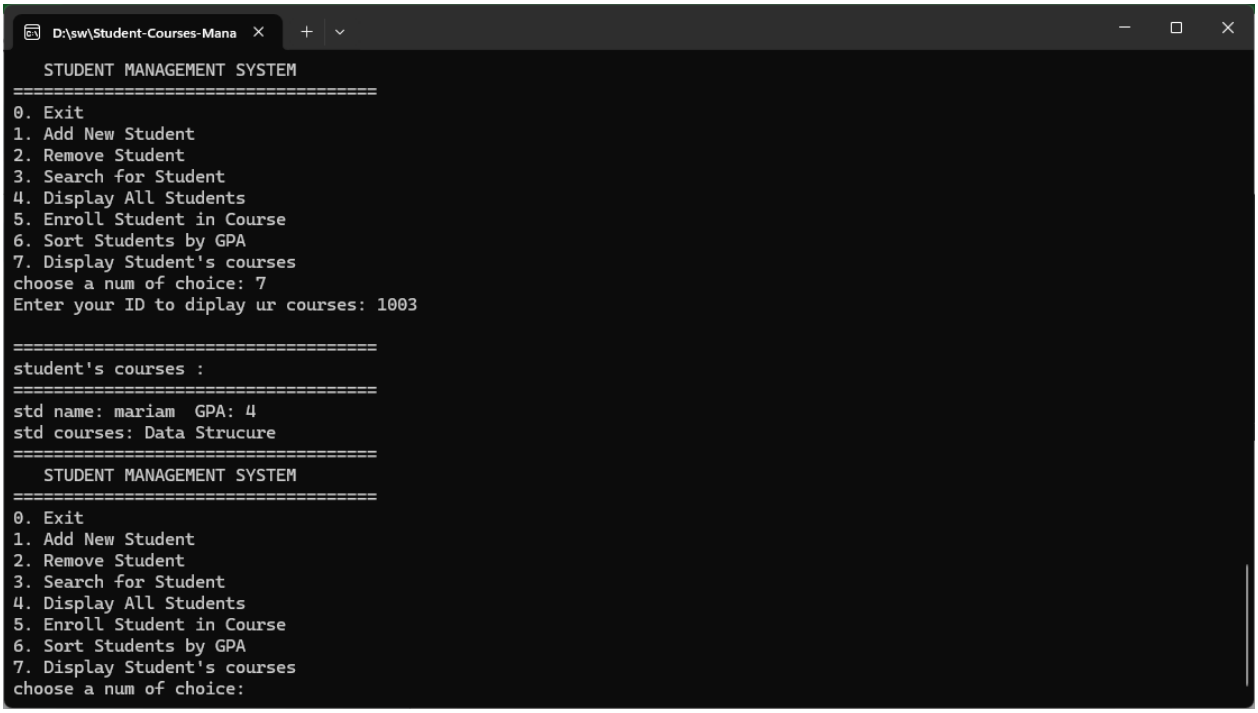
10.4 Displaying All Students



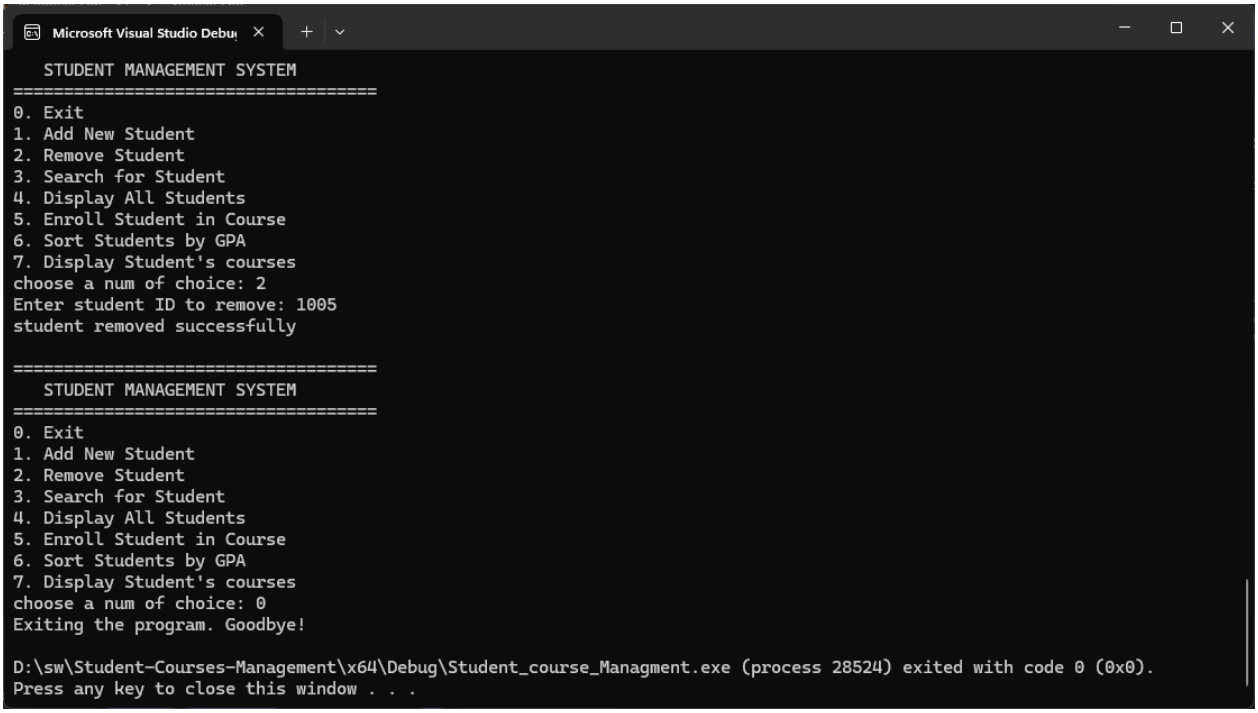
10.5 Sorting by GPA



10.6 Display student’s courses



10.7 Remove student and exit



11. Conclusion

This project successfully demonstrates the implementation of a Student Course Management System using C++ with a focus on Object-Oriented Programming principles and the Standard Template Library. The system meets all the functional requirements defined in the assignment: adding, removing, searching, and displaying students; enrolling students in predefined courses; preventing duplicate enrollments; validating GPA input; sorting by GPA; and persisting data between sessions through file handling.

Through building this project, several important programming skills were practiced. Designing a class hierarchy with inheritance, choosing the right STL containers for different purposes, writing modular functions, handling file I/O, and debugging parsing logic were all challenges that led to a deeper understanding of how C++ programs are structured in practice.

The project also highlighted some areas for future improvement. The current text-file storage approach is simple but not scalable. A database system would be more appropriate for larger datasets. The lack of an update or edit feature means that a student's name or GPA cannot be changed after they are added, which would be a useful addition. Additionally, adding a graphical user interface would make the system more accessible to non-technical users.

Overall, this assignment provided a solid foundation in applying C++ concepts to solve a realistic problem, and the result is a functional, well-organized application that reflects the skills developed throughout the course.

12. References

[1] B. Stroustrup, The C++ Programming Language, 4th ed. Addison-Wesley, 2013.

[2] cppreference.com – STL Containers. Available: https://en.cppreference.com/w/cpp/container

[3] stackoverflow.com – STL Algorithms. Available: https://stackoverflow.com/questions/23823861/using-the-find-if-function

https://www.w3schools.com/cpp/cpp_functions_lambda.asp

[4] cppreference.com – File I/O (fstream). Available: https://en.cppreference.com/w/cpp/io/basic_fstream

[5] GitHub Repository – AmiraDeef/Student-Courses-Management. Available: https://github.com/AmiraDeef/Student-Courses-Management